Objetos, Spread, desestructuración y colecciones de datos

Objetivo: Conocer la ruta de la clase y lo que podremos construir al terminar.

Teoría

objetos, JSON, desestructuración, spread, arreglos, Set y Map.

Práctica

checkpoints, ejercicios guiados, trabajo en casa y reto integrador.

Meta

organizar, consultar y transformar datos de casos reales.



Objetos: datos con nombre

Objetivo: Crear un objeto y leer sus propiedades con el operador punto.

Un objeto agrupa datos relacionados en pares clave: valor, separados por comas.

Los valores pueden ser de cualquier tipo, incluidos arreglos y otros objetos.

Se accede con punto: objeto.clave.

```
const persona = { nombre: "Dani", familia: ["Miguel", "María"] };
const casa = { direccion: { ciudad: "Barcelona" } };
console.log(persona.nombre, persona.familia[0]);
console.log(casa.direccion.ciudad);
```

Salida:

```
Dani Miguel
Barcelona
```

Métodos y this

Objetivo: Definir métodos y entender a qué objeto apunta this.

Un método es una función asociada a una clave; se ejecuta con ().

La sintaxis recomendada omite la palabra function.

```
const persona = {
  nombre: "Juan",
  saludar() {
    return "Hola, soy " + this.nombre;
  }
};
console.log(persona.saludar());
```

Salida:

```
Hola, soy Juan
```

Dónde se usa	Qué es this
Método de objeto	El objeto que lo llama
Fuera de funciones, en un script normal	window
Función normal	window; undefined en modo estricto
Evento	El elemento HTML que lo recibió
call(), apply(), bind()	El objeto que se les indique

🤔 Pregunta rápida: objetos

Objetivo: Comprobar la lectura de propiedades, métodos y this.

1

¿Qué imprime este código?

```
const p = { nombre: "Ana" };
console.log(p.edad);
```

2

¿Qué imprime este código?

```
const caja = {
  valor: 5,
  doble() { return this.valor * 2; }
};
console.log(caja.doble());
```

3

¿Qué diferencia hay entre una propiedad y un método?

JSON: datos como texto

Objetivo: Reconocer qué es JSON y sus reglas de sintaxis.

Formato ligero para intercambiar datos, independiente del lenguaje.

Se usa entre aplicaciones y servidores, en archivos .json y en configuraciones.

Claves y strings siempre con comillas dobles. Números y booleanos, sin comillas.

{} contiene objetos y [] contiene arreglos.

Para verlo gráficamente en VS Code: extensión JSON Crack.

```
const texto = '{"nombre":"Ana","edad":30}';
console.log(typeof texto);
```

Salida:

```
string
```

JSON.stringify(): de objeto a texto

Objetivo: Convertir objetos en JSON y filtrar lo que se incluye.

Sintaxis: JSON.stringify(valor, replacer, space).

replacer: arreglo de claves permitidas o función que transforma valores.

space: espacios de indentación, por ejemplo 2 o 4.

```
const persona = { nombre: "Juan", clave: "1234" };
const ocultar = (key, value) =>
  (key === "clave") ? "****" : value;
console.log(JSON.stringify(persona));
console.log(JSON.stringify(persona, ["nombre"]));
console.log(JSON.stringify(persona, ocultar));
```

Salida:

```
{"nombre":"Juan","clave":"1234"}
{"nombre":"Juan"}
{"nombre":"Juan","clave":"****"}
```

JSON.parse(): de texto a objeto

Objetivo: Convertir texto JSON en un objeto que podamos usar.

> Sintaxis: JSON.parse(text, reviver). Si el texto no es JSON válido, se produce un error.

> **reviver:** función opcional que transforma cada valor durante la conversión.

```
const texto = '{"nombre":"Juan","edad":30}';
const persona = JSON.parse(texto);
console.log(persona.nombre, persona.edad + 1);
```

Salida:

```
Juan 31
```

```
const mayus = (key, value) =>
  (typeof value === "string") ? value.toUpperCase() : value;
console.log(JSON.parse('{"nombre":"Juan","edad":30}', mayus));
```

Salida:

```
{nombre: 'JUAN', edad: 30}
```

🤔 Pregunta rápida: JSON

Objetivo: Distinguir entre texto JSON y objeto JavaScript.

1

¿Qué imprime este código?

```
const texto = '{"a":1}';
console.log(texto.a);
```

2

¿Qué imprime este código?

```
const dato =
JSON.parse('{"precio":"5"}');
console.log(dato.precio + 1);
```

3

¿Qué imprime este código?

```
const obj = { a: 1, hola() { return
"hola"; } };
console.log(JSON.stringify(obj));
```

Desestructuración de objetos

Objetivo: Extraer propiedades a variables con alias, valores por defecto y anidación.

Básica: { nombre } crea la variable nombre.

Alias: { ciudad: lugar } crea lugar, no ciudad.

Por defecto: { edad = 30 } aplica si la propiedad es undefined.

Anidada: { dir: { cp } } entra al objeto interno.

```
const persona = {
  nombre: "Juan",
  ciudad: "Madrid",
  dir: { cp: "28001" }
};
const {
  nombre,
  ciudad: lugar,
  edad = 30,
  dir: { cp }
} = persona;
console.log(nombre, lugar, edad, cp);
```

Salida:

```
Juan Madrid 30 28001
```

Desestructuración de arreglos y rest

Objetivo: Extraer elementos por posición y agrupar lo restante.

> En arreglos importa la posición. Una coma extra omite un elemento.

> Rest (...) agrupa lo que falta en un arreglo u objeto nuevo.

```
const numeros = [1, 2, 3, 4, 5];
const [primero, , tercero, ...resto] = numeros;
console.log(primero, tercero, resto);
```

Salida:

```
1 3 (2) [4, 5]
```

Ese (2) no es parte del dato: es la forma en que la consola de Chrome te indica cuántos elementos tiene el arreglo. `resto` contiene solo [4, 5], y como son 2 elementos, la consola lo muestra como (2) [4, 5]

- `nombre`: guarda `"Juan"`, porque se pidió por su nombre.
- `datos`: es un objeto nuevo con todo lo que no se pidió por nombre, es decir, `edad` y `pais`. Eso hace el rest (...).
- Por eso, para leer la edad con ese código hay que pasar por `datos`:

```
console.log(datos.edad); // 25
```

```
const persona = { nombre: "Juan", edad: 25, pais: "España" };
const { nombre, ...datos } = persona;
console.log(nombre, datos);
```

Salida:

```
Juan {edad: 25, pais: 'España'}
```

Spread y clonación

Objetivo: Combinar y copiar datos, y conocer el límite de la copia.

> **Spread (...):** expande elementos de un iterable. Lo último que se escribe sobrescribe propiedades existentes.

> **Copia superficial:** tanto `Object.assign({}, origen)` como el operador spread `(...)` copian solo el primer nivel del objeto, no los objetos anidados.

los corchetes de afuera crean un solo arreglo nuevo, y el spread `(...)` saca los elementos de `[1, 2]` y los coloca sueltos dentro de él:

```
const persona = { nombre: "Juan", edad: 25 };
const info = { ...persona, edad: 26 };
console.log(info, [...[1, 2], 3]);
```

Salida:

```
{nombre: 'Juan', edad: 26} (3) [1, 2, 3]
```

```
const original = { nombre: "Ana", dir: { ciudad: "Cali" } };
const copia = Object.assign({}, original);
copia.nombre = "Eva";
copia.dir.ciudad = "Pasto"; // Modifica el objeto anidado en original
console.log(original.nombre, original.dir.ciudad);
```

Salida:

Por eso, cambiar `nombre` en la copia no afecta al original, pero cambiar `ciudad` sí:

```
Ana Pasto
```

🤔 Pregunta rápida: extraer y copiar

Objetivo: Razonar sobre alias, valores por defecto y copias.

1

¿Qué imprime este código?

```
const { nombre: alias } = { nombre:
"Ana" };
console.log(nombre);
```

2

¿Qué imprime este código?

```
const [a, b = 9] = [1];
console.log(a, b);
```

3

¿Qué imprime este código?

```
const x = { a: 1 };
const y = { ...x, a: 5 };
console.log(x.a, y.a);
```

Colecciones y manipulación de arreglos

Objetivo: Reconocer qué es una colección y modificar un arreglo con sus métodos básicos.

Colección: grupo de datos relacionados. En JavaScript veremos Array, Set y Map.

Con const, el arreglo no se reasigna, pero su contenido sí puede cambiar.

push devuelve la nueva longitud. pop devuelve el elemento eliminado.

splice(índice, nEliminar, ...nuevos) modifica el arreglo y devuelve lo eliminado.

join(sep) y toString() crean un string o arman un texto con los elementos sin modificar el arreglo.

```
const frutas = ["Manzana", "Banana", "Naranja"];
console.log(frutas.push("Pera"));
console.log(frutas.pop());
frutas.splice(1, 1, "Kiwi");
console.log(frutas.join(", "));
```

```
frutas.splice(1, 1, "Kiwi");
//            | |  └── qué poner en ese lugar
//            | └──────── cuántos elementos quitar
//            └────────────── desde qué posición empezar
```

Salida:

```
4
Pera
Manzana, Kiwi, Naranja
```

Buscar en arreglos

Objetivo: Encontrar elementos por valor exacto o por condición.

Método	Busca	Si no encuentra
indexOf / lastIndexOf	Posición de un valor	-1
find	Primer elemento que cumple	undefined
findIndex	Posición del primero que cumple	-1

```
const frutas = ["Manzana", "Banana", "Manzana"];
console.log(
  frutas.indexOf("Manzana"),
  frutas.lastIndexOf("Manzana")
);
```

Salida:

```
0 2
```

```
const edades = [25, 30, 18, 15];
console.log(edades.find(edad => edad < 20));
console.log(edades.findIndex(edad => edad < 20));
```

edad => edad < 20 es una función corta: recibe cada elemento y devuelve la condición.

Salida:

```
18
2
```

Ordenar arreglos

Objetivo: Ordenar texto y números sin resultados inesperados.

> sort() compara como texto si no recibe función.

> Con números se usa (a, b) => a - b (ascendente) o b - a (descendente).

> sort() y reverse() modifican el arreglo. Copia con spread si necesitas conservarlo.

`[...numeros]` crea un arreglo nuevo con los mismos elementos, y `sort()` se aplica sobre ese arreglo nuevo, no sobre `numeros`.

```
const numeros = [10, 5, 8, 2, 1];
console.log([...numeros].sort());
console.log([...numeros].sort((a, b) => a - b));
console.log([...numeros].sort((a, b) => b - a));
console.log(["Rojo", "Verde", "Azul"].reverse());
```

Salida:

```
(5) [1, 10, 2, 5, 8]
(5) [1, 2, 5, 8, 10]
(5) [10, 8, 5, 2, 1]
(3) ['Azul', 'Verde', 'Rojo']
```

🤔 Pregunta rápida: arreglos

Objetivo: Anticipar qué devuelve cada método y qué modifica.

1

¿Qué imprime este código?

```
const n = [10, 9, 1];
console.log(n.sort());
```

2

¿Qué devuelve find si nada cumple la condición? ¿Y findIndex?

Set: valores únicos

Objetivo: Guardar valores sin repetidos y consultarlos.

add(valor), delete(valor), has(valor), clear() y la propiedad size.

Agregar un valor repetido se ignora. 2 y "2" son valores distintos.

Spread convierte un Set en arreglo.

```
const miSet = new Set([1, 2, 2, "2"]);
miSet.add(3);
console.log(miSet.size, miSet.has(2), miSet.delete(9));
console.log([...miSet]);
```

Salida:

```
4 true false
(4) [1, 2, '2', 3]
```





Map: claves de cualquier tipo

Objetivo: Asociar claves con valores y recuperar datos por clave.

set, get, has, delete, clear y la propiedad size.

La clave puede ser de cualquier tipo, incluso un objeto.

Si la clave es un objeto, usa la misma variable en set y en get. Sino dara undefined

Cuando la clave es un objeto, Map no compara su contenido, sino si es **exactamente el mismo objeto en memoria**. Es decir, compara la referencia:

```
const mapa = new Map();
const id = { nombre: "Juan" };
mapa.set(id, { edad: 23 });
console.log(mapa.get(id),
mapa.size);
console.log(mapa.get({ nombre:
"Juan" }));
```

Salida:

```
{edad: 23} 1
undefined
```

- `id` es el objeto que se usó en `set`, así que `get(id)` lo encuentra.
- `{ nombre: "Juan" }` es otro objeto, creado en ese momento, así que `get` no encuentra esa clave y devuelve `undefined`.

🤔 Pregunta rápida: Set y Map

Objetivo: Comprobar la unicidad en Set y las claves en Map.

1

¿Qué imprime este código?

```
const s = new Set([1, "1", 1]);
console.log(s.size);
```

2

¿Cuándo usarías un Set y cuándo un Map?

Chuleta: ¿qué necesito y qué uso?

Objetivo: Relacionar problemas típicos con la herramienta adecuada.

Necesito...	Uso
Agrupar datos y acciones	Objeto y métodos
Enviar o guardar un objeto como texto	JSON.stringify()
Leer texto JSON	JSON.parse()
Sacar datos a variables	Desestructuración
Copiar o combinar	Spread, Object.assign()
Buscar por condición	find, findIndex
Buscar por valor	indexOf, lastIndexOf
Ordenar números	sort((a, b) => a - b)
Quitar repetidos	Set
Consultar por clave	Map

Práctica: orden y reglas de juego

Objetivo: Conocer cómo vamos a trabajar y qué se revisará.

Orden: ✅ checkpoints → 🏫 guiados → 🏠 casa → 🔥 reto

Usa index.html + js/main.js, con el script justo antes de </body>.

No modifiques datos ni código base, salvo donde el ejercicio lo indique.

Comenta cada requisito con // R1.

Usa const por defecto y let solo si hay que reasignar. No uses var.

Compara con === y !==.

No uses librerías externas.

Verifica cada resultado en la consola.

✅ Checkpoint 1: ficha de estudiante

Objetivo: Combinar objetos, this y desestructuración.

Columna 1 — Retos

1. Imprime estudiante.saludar().
2. Desestructura nombre como alumna y ciudad con "Cali" por defecto.
3. Imprime nota + 1 y Number(nota) + 1. Explica por qué difieren.
4. Desestructura beca con "sin beca" por defecto. Explica el resultado.
5. Desestructura saludar y llámalo suelto. Explica por qué cambia.

Columna 2 — Código base

```
const estudiante = {
 nombre: "Laura",
 nota: "4",
 beca: null,
 saludar() {
 return "Hola, soy " + this.nombre;
 }
};
```

✅ Checkpoint 2: pedido en línea

Objetivo: Combinar JSON, comparaciones y spread.

Columna 1 — Retos

1. Convierte textoPedido en pedido. Imprime typeof de ambos.
2. Compara pedido.total con 45000 usando === y ==, y luego Number(...) === 45000. Explica.
3. Crea copiaPedido con spread y cliente: "Ana". Cambia copiaPedido.envio.ciudad a "Ipiales". Imprime cliente y ciudad de pedido. Explica.
4. Imprime un JSON de pedido solo con cliente y total.

Columna 2 — Código base

```
const textoPedido =
 '{"cliente":"Mario","total":"45000",
 "envio":{"ciudad":"Pasto"}}';
```

✅ Checkpoint 3: ranking del torneo

Objetivo: Combinar sort, spread, Set y búsqueda.

Columna 1 — Retos

1. Copia puntajes en ranking y usa sort() sin función. Explica el orden.
2. Ordena ranking de mayor a menor. Comprueba que puntajes no cambió.
3. Crea un Set. Imprime size, has(100) y has("100").
4. Imprime indexOf(100) y lastIndexOf(100). Busca un puntaje mayor que 200 con find y findIndex. Explica.

Columna 2 — Código base

```
const puntajes = [100, 25, 9, 100, 7];
```

🏫 Guiado 1: tienda La Esquina

Objetivo: Consultar y transformar un inventario recibido como JSON.

Columna 1 — Requisitos

- **R1:** Convierte a productos. Imprime el typeof de un precio.
- **R2:** Busca "Café" y desestructura sus datos.
- **R3:** Imprime precio + 2, precio * 2 y Number(precio) * 2.
- **R4:** Imprime el primer producto agotado.
- **R5:** Ordena una copia por stock descendente. Imprime el primero de cada arreglo.
- **R6:** Crea cafeNuevo con precio "13000". Compara ambos precios.
- **R7:** Imprime cafeNuevo en JSON con stock como "reservado".

Salida esperada

```
string
Café 12000 4
120002 24000 24000
Panela
Café Arroz
12000 13000
{"nombre":"Café","precio":"13000","stock":"reservado"}
```

Columna 2 — Código base

```
const inventarioJSON = '[' +
 '{"nombre":"Arroz","precio":"3200","stock":3},' +
 '{"nombre":"Panela","precio":"2500","stock":0},' +
 '{"nombre":"Café","precio":"12000","stock":4}' +
 ']';
```

🏫 Guiado 2: taller de robótica

Objetivo: Depurar inscripciones y gestionar cupos con Set y Map.

Columna 1 — Requisitos

- **R1:** Crea el Set unicos. Imprime size.
- **R2:** Pásalo a lista, ordénala e imprímela con join(" | ").
- **R3:** Crea salones con la clave salonA y el valor { cupos: 2 }.
- **R4:** Usa get(salonA) y get({ nombre: "A" }). Explica.
- **R5:** Retira a Sara con indexOf y splice. Imprime lo retirado y lista.
- **R6:** Crea asistencia, registra lista[0] e imprime dos has y size.

Salida esperada

```
3
ana@mail.co | luis@mail.co | sara@mail.co
{cupos: 2}
undefined
(1) ['sara@mail.co']
(2) ['ana@mail.co', 'luis@mail.co']
true false 1
```

Columna 2 — Código base

```
const inscritos = [
 "sara@mail.co",
 "ana@mail.co",
 "luis@mail.co",
 "ana@mail.co",
 "sara@mail.co"
];
const salonA = { nombre: "A" };
```

🏠 Para la casa

Objetivo: Reforzar de forma autónoma los temas combinados en clase.

Caso	Temas que combina
Factura de la papelería	Objetos, métodos, this, desestructuración, valores por defecto, NaN
Notas del curso	sort, spread, find, findIndex, Set, splice, join
Agenda de la veterinaria	JSON, Map, Object.assign, copia superficial, spread

El enunciado completo está en el README.

🔥 Reto integrador: biblioteca escolar

Objetivo: Combinar la mayoría de los temas en un programa completo.

Req.	Qué debe hacer	Evalúa
R1	Convertir el catálogo e imprimir un typeof	JSON.parse, tipos
R2	Crear biblioteca con buscar(codigo)	Métodos, this
R3	Depurar solicitudes	Set, spread
R4	Atender 3 pedidos; informar "No existe"	find, undefined
R5	Ordenar una copia por páginas	sort, conversión
R6	Prestar una copia con disponible: false	Map, spread
R7	Agregar préstamo y consultar el Map	has, size
R8	Reportar en JSON con paginas oculta	Replacer

```
const catalogoJSON = '[' +
 '{"codigo":"L1","titulo":"María","paginas":"320","disponible":true},' +
 '{"codigo":"L2","titulo":"La vorágine","paginas":"256","disponible":true},' +
 '{"codigo":"L3","titulo":"El túnel","paginas":"98","disponible":true}' +
 ']';
const solicitudes = ["L3", "L9", "L1", "L3", "L9"];
```

🔥 Reto: entregable y rúbrica

Objetivo: Conocer qué se entrega y cómo se evalúa.

Entregable: carpeta biblioteca/ con index.html y js/main.js, comprimida en .zip.

Criterio	Requisitos	Peso
Datos y JSON	R1, R8	20%
Objetos, métodos y this	R2	20%
Colecciones: Set y Map	R3, R6, R7	25%
Búsqueda y manejo de undefined	R4	15%
Orden sin mutar el original	R5	10%
Reglas de código: // R1, const, ===, sin var	Todos	10%

Resumen y antes de entregar

Objetivo: Repasar las ideas clave y revisar el trabajo antes de enviarlo.

Resumen

Objetos agrupan datos y acciones. JSON los transporta como texto.

Desestructuración y spread extraen, combinan y copian datos.

Arreglos, Set y Map organizan colecciones según la necesidad.

Antes de entregar

- El script está vinculado justo antes de </body>.
- Cada requisito tiene su comentario // R1.
- Uso const o let, nunca var.
- Comparo con === y !==.
- No modifiqué los datos ni el código base sin que se pidiera.
- La consola no muestra errores y la salida coincide con la esperada.

